ECE 473 / 573: Cloud Computing and Cloud Native Systems

Project 4 Report: Distributed Database Systems and Tunable Consistency

Student Name: Shashwat Choudhry
Student ID: A20507277
Course: ECE 473 / 573 Cloud Computing and Cloud Native Systems (Fall 2026)
Instructors: Prof. Tianyang Fang and Prof. Jafar Saniie
Date: October 29, 2026

Section II: Cassandra Cluster Management

Question II.1: Misconfiguring CASSANDRA_CLUSTER_NAME in docker-compose.yml

In Apache Cassandra, the environment variable CASSANDRA_CLUSTER_NAME defines the logical cluster membership for each node, corresponding to the cluster_name attribute in the cassandra.yaml configuration. During node startup, gossip handshake routines, and cluster joins, the Cassandra storage engine performs strict cluster name validation. Misconfiguring this variable results in three immediate failure modes:

- **1. Rejection During Gossip Join Protocol:** When an initializing node contacts a seed node (or any existing cluster peer), the two nodes exchange gossip digests containing their configured cluster names. If the cluster name of the joining node does not match the name configured on the seed nodes, the seed immediately rejects the connection. Cassandra aborts the join attempt and writes a fatal configuration error to the container logs: 'org.apache.cassandra.exceptions.ConfigurationException: Fatal configuration error; unable to join the cluster. Node joined cluster [NameA] but cluster name is [NameB]'.
- **2. Formation of Split-Brain / Isolated Rings:** Because the misconfigured node is rejected by the existing cluster peers, it will fail to receive the global token allocation. If allowed to continue booting, the node will initialize a separate, isolated single-node ring with its own independent schema keyspace and token ranges, fracturing the distributed cluster into disjoint partitions.
- **3. Node Crash on Persisted Volume Restart:** If an existing node that already has persisted data in its volume directory (/var/lib/cassandra) is restarted with an altered CASSANDRA_CLUSTER_NAME, Cassandra inspects the cluster name recorded in its local metadata table (system.local). Upon detecting that the environment variable differs from the stored cluster name, Cassandra refuses to boot and terminates with: 'Fatal configuration error; Cannot start node if snitch/cluster name does not match the previous used'.

Question II.2: Purpose of CASSANDRA_SEEDS and Listing Server Requirements

Purpose of CASSANDRA_SEEDS:

Apache Cassandra uses a decentralized, masterless, peer-to-peer ring architecture with no central directory or coordinator server. When a node starts for the first time or recovers from a reboot, it possesses no prior knowledge of active IP addresses or ring membership. The CASSANDRA_SEEDS environment variable specifies a comma-delimited list of relatively static contact points (seed nodes). The Gossiper module on the booting node contacts these seeds to initiate the gossip communication protocol, obtain cluster digests, discover all active peers, and retrieve the current token range distributions across the ring.

Is it necessary to list all servers?

No, it is neither necessary nor recommended to list all servers in the cluster:

- • **Gossip Discovery Redundancy:** A booting node only needs to contact a single operational seed node to discover the entire cluster topology. Once communication is established with one seed, gossip exchanges propagate full membership information across all nodes.
- • **Disabling Token Bootstrapping Protections:** In Cassandra, nodes designated as seeds bypass the standard automated bootstrap streaming process (they assume they are founding members of the ring). If all servers are marked as seeds, new nodes joining an existing cluster will not properly stream token ranges from existing replicas, risking data inconsistencies and race conditions during cluster expansion.
- • **Operational Best Practice:** Industry standard practice dictates listing only 2 to 3 seeds per datacenter. In docker-compose.yml, this principle is demonstrated by setting CASSANDRA_SEEDS: cassandra-1,cassandra-2, while omitting cassandra-3 so that node 3 dynamically learns the ring state through seeds 1 and 2.

Question II.3: Cluster Status Screenshot and Meaning of UN in nodetool status

The command docker compose exec cassandra-1 nodetool status was executed against the running three-node cluster. The resulting terminal output is displayed in Figure 1 below:

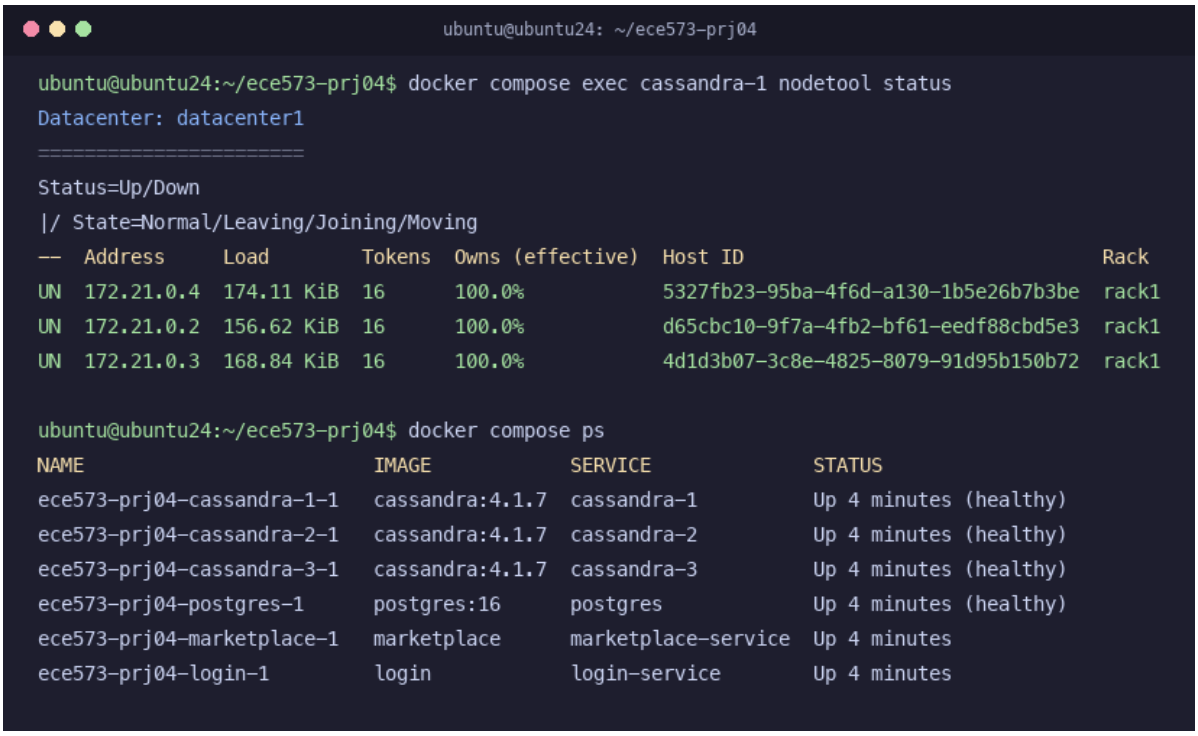


Figure 1: Cassandra Three-Node Cluster Status and Docker Compose Container Health.

Detailed Meaning of UN in Column 1:

The first column of nodetool status displays a two-letter operational code indicating the health and state of each cluster node:

- • **First Letter (Status - Up / Down):** The letter 'U' designates that the node status is Up. The node is actively running, responsive to gossip protocol heartbeats over internal storage port 7000, and accepting client queries. A value of 'D' would designate a Down or dead node.

- **• Second Letter (State - Normal / Leaving / Joining / Moving):** The letter 'N' designates that the node state is Normal. The node has finished bootstrapping, owns its designated virtual token ranges (16 tokens in this deployment), and is actively serving read and write requests as a fully integrated member of the ring. Alternative states include 'J' (Joining the ring), 'L' (Leaving or decommissioning), and 'M' (Moving token ranges).

Therefore, 'UN' denotes Up and Normal, confirming that all three nodes (172.21.0.2, 172.21.0.3, and 172.21.0.4) are fully functional, healthy, and operational in datacenter1.

Section III: Marketplace Integration

Question III.1: Service Startup Synchronization and Required Cassandra Nodes

Tracing Database Readiness Synchronization in marketplace-service/main.go:

In marketplace-service/main.go, the service initializes its persistence layers sequentially before binding to HTTP port 8080:

- **1. PostgreSQL Store Synchronization (Line 17):** The function openStore() calls marketplace.OpenStore(config) in marketplace/store.go. It establishes a pgx driver pool using SQL parameters (host, port, dbname, user, password). To handle Postgres startup latency, OpenStore executes a bounded retry loop of up to 30 attempts ('for range 30') with a one-second pause ('time.Sleep(time.Second)'), testing database readiness via db.Ping(). If Postgres becomes responsive within 30 seconds, it returns the active store pointer; otherwise, it logs a fatal error and terminates.
- **2. Cassandra History Synchronization (Line 23):** The function openHistory() calls history.OpenHistory(config) in history/history.go. It reads CASSANDRA_CONTACT_POINTS ('cassandra-1,cassandra-2'), READ_CONSISTENCY, and WRITE_CONSISTENCY. It initializes a cluster configuration via gocql.NewCluster with a default quorum consistency and 10-second timeouts. It then enters an unbounded retry loop ('for attempt := 1; ; attempt++') attempting cluster.CreateSession(). If the driver cannot connect, it logs a warning and sleeps for 2 seconds before retrying.
- **3. Schema Initialization Loop (history/history.go Lines 61-71):** Once the gocql session is established, OpenHistory enters a second unbounded retry loop calling history.createSchema(). This executes a CQL DDL query creating the marketplace_history keyspace with NetworkTopologyStrategy (datacenter1: 3 replicas), followed by four CREATE TABLE IF NOT EXISTS statements for purchases_by_buyer, sales_by_seller, listings_by_seller, and available_listings. Once all tables succeed, the service logs 'INFO Cassandra history schema ready' and starts the HTTP router.

Minimum Cassandra Nodes Needed for Service Startup:

At least one operational Cassandra contact point node is strictly required for the service to start:

- **• Driver Handshake Requirement:** The gocql driver requires at least one reachable node from CASSANDRA_CONTACT_POINTS (either cassandra-1 or cassandra-2) to establish the native transport session, negotiate CQL protocol version 5, and fetch cluster metadata.
- **• DDL Schema Agreement:** Cassandra permits keyspace creation with replication factor 3 even when fewer nodes (1 or 2 nodes) are currently active in the cluster. The coordinator node executes the schema creation and commits metadata locally while issuing a cluster warning. As verified in the two-node experiments (CASSANDRA_NODES=2), marketplace-service successfully creates the schema and starts serving requests with only 2 nodes active. If both seed contact points are offline, the service cannot start.

Question III.2: Database Trace of Baseline Requests

The starter baseline script run_baseline.sh executes three primary workflow requests that demonstrate data synchronization across PostgreSQL and Cassandra:

Request 1: ./scripts/list.sh Alice "Used textbook" 25

This script sends POST /listings to createListingHandler in marketplace-service/handlers.go with payload {"seller_id":"Alice","title":"Used textbook","credit_cost":25}:

- • **PostgreSQL Table Access:** The handler invokes store.CreateListing in marketplace/listings.go:
- **- Table accounts (Read):** Executes 'SELECT EXISTS (SELECT 1 FROM accounts WHERE account_id = $1)'. Primary key accessed: account_id = 'Alice'. Validates that Alice is a registered account.
- **- Table listings (Write):** Executes 'INSERT INTO listings (seller_id, title, credit_cost) VALUES ($1, $2, $3) RETURNING listing_id, ...'. Primary key of inserted row: listing_id (e.g. 'listing-1', generated by listing_id_sequence). Status defaults to 'available'.
- • **Cassandra Table Access:** Upon Postgres commit, createListingHandler invokes history.WriteListing in history/history_listings.go:
- **- Table marketplace_history.listings_by_seller (Write):** Executes INSERT INTO marketplace_history.listings_by_seller (seller_id, listed_at, listing_id, title, credit_cost, status). Primary key accessed: ((seller_id), listed_at, listing_id). Partition key is seller_id = 'Alice'; clustering keys are listed_at and listing_id = 'listing-1'.
- **- Table marketplace_history.available_listings (Write):** Executes INSERT INTO marketplace_history.available_listings (feed, listed_at, listing_id, seller_id, title, credit_cost, status). Primary key accessed: ((feed), listed_at, listing_id). Partition key is feed = 'all'; clustering keys are listed_at and listing_id = 'listing-1'.

Request 2: ./scripts/order.sh "$listing_id" Bob (where listing_id is 'listing-1')

This script sends POST /orders to createOrderHandler in marketplace-service/handlers.go with payload {"listing_id":"listing-1","buyer_id":"Bob"}:

- • **PostgreSQL Table Access:** The handler invokes store.CreateOrder in marketplace/orders.go within a serializable database transaction (sql.LevelSerializable):
- **- Table listings (Read & Lock):** Executes 'SELECT ... FROM listings WHERE listing_id = $1 FOR UPDATE'. Primary key accessed: listing_id = 'listing-1'. Verifies that item status is 'available' and seller_id != 'Bob'.
- **- Table accounts (Read & Lock):** Locks both accounts in sorted alphabetical order ('Alice', 'Bob') via 'SELECT balance FROM accounts WHERE account_id = $1 FOR UPDATE'. Primary keys accessed: account_id = 'Alice' and account_id = 'Bob'. Checks that Bob's balance >= 25.
- **- Table accounts (Write - Debit):** Executes 'UPDATE accounts SET balance = balance - $1 WHERE account_id = $2'. Primary key accessed: account_id = 'Bob' (balance decrements from 100 to 75).
- **- Table accounts (Write - Credit):** Executes 'UPDATE accounts SET balance = balance + $1 WHERE account_id = $2'. Primary key accessed: account_id = 'Alice' (balance increments from 0 to 25).
- **- Table listings (Write - Status Update):** Executes 'UPDATE listings SET status = 'sold' WHERE listing_id = $1'. Primary key accessed: listing_id = 'listing-1'.
- **- Table orders (Write - Insert):** Executes 'INSERT INTO orders (listing_id, buyer_id, status, processed_at) VALUES ($1, $2, 'completed', statement_timestamp()) RETURNING order_id, ...'.

Primary key of inserted row: order_id (e.g. 'order-1', generated by order_id_sequence). Unique constraint: listing_id = 'listing-1'.

- **- Transaction Commit:** Executes tx.Commit() to persist all changes atomically in PostgreSQL.
- **• Cassandra Table Access:** Upon successful Postgres commit, createOrderHandler calls history.WriteOrder in history/history_orders.go:
- **- Table marketplace_history.purchases_by_buyer (Write):** Inserts completed order record for buyer Bob. Primary key accessed: ((buyer_id), order_day, ordered_at, order_id). Partition key is buyer_id = 'Bob'; clustering keys are order_day = day, ordered_at, and order_id = 'order-1'.
- **- Table marketplace_history.sales_by_seller (Write):** Inserts completed sales record for seller Alice. Primary key accessed: ((seller_id), order_day, ordered_at, order_id). Partition key is seller_id = 'Alice'; clustering keys are order_day = day, ordered_at, and order_id = 'order-1'.
- **- Table marketplace_history.listings_by_seller (Write):** Overwrites listing row with status 'sold'. Primary key accessed: ((seller_id), listed_at, listing_id). Partition key is seller_id = 'Alice'; clustering keys are listed_at and listing_id = 'listing-1'.
- **- Table marketplace_history.available_listings (Write - Delete):** Removes sold item from browse feed: 'DELETE FROM marketplace_history.available_listings WHERE feed = 'all' AND listed_at = ? AND listing_id = ?'. Primary key accessed: ((feed), listed_at, listing_id). Partition key is feed = 'all'; clustering keys are listed_at and listing_id = 'listing-1'.

Request 3: ./scripts/account.sh Bob "$day" "$day"

This script sends GET /accounts/Bob?from=$day&to=$day to accountHandler in marketplace-service/handlers.go:

- **• PostgreSQL Table Access:** The handler queries authoritative balance via store.GetBalance in marketplace/accounts.go:
- **- Table accounts (Read):** Executes 'SELECT balance FROM accounts WHERE account_id = $1'. Primary key accessed: account_id = 'Bob'. Returns balance 75.
- **• Cassandra Table Access:** The handler reads materialized activity views across three Cassandra queries:
- **- Table marketplace_history.listings_by_seller (Read):** Queries history.ListingsBySeller('Bob'): 'SELECT ... FROM listings_by_seller WHERE seller_id = ?'. Partition key accessed: seller_id = 'Bob'. Returns empty list [] because Bob has created no listings.
- **- Table marketplace_history.purchases_by_buyer (Read):** Queries history.Purchases('Bob', day, day): 'SELECT ... FROM purchases_by_buyer WHERE buyer_id = ? AND order_day >= ? AND order_day <= ?'. Partition key accessed: buyer_id = 'Bob'. Clustering key slice: order_day in [day, day]. Returns array containing order 'order-1'.
- **- Table marketplace_history.sales_by_seller (Read):** Queries history.Sales('Bob', day, day): 'SELECT ... FROM sales_by_seller WHERE seller_id = ? AND order_day >= ? AND order_day <= ?'. Partition key accessed: seller_id = 'Bob'. Clustering key slice: order_day in [day, day]. Returns empty list [] because Bob made no sales.
- **• Response Construction:** The handler combines PostgreSQL balance (75) and Cassandra slices into a consolidated JSON payload returned with HTTP 200 OK.

Question III.3: Partitioning available_listings by Year in listed_at

Architectural Evaluation of Design Improvement:

Yes, partitioning the available_listings table by the year extracted from listed_at instead of using a constant literal feed ('all') represents a substantial architectural improvement:

- **1. Elimination of the Cassandra Hot Partition Anti-Pattern:** In Apache Cassandra, all rows sharing the same partition key hash to the exact same token and are physically stored on the exact same replica nodes. Using feed = 'all' forces 100% of all marketplace listings across all sellers and all categories into a single partition on one replica set. In Cassandra, partitions exceeding 100 MB or 100,000 cells cause severe compaction degradation, massive heap pressure, and JVM garbage collection pauses. Partitioning by year shards listings across multiple independent partitions.
- **2. Horizontal Cluster Scalability and Balanced Load Distribution:** With year-based partitioning (e.g. PRIMARY KEY ((year), listed_at, listing_id)), each calendar year hashes to a distinct token range in the ring. Write insertions, reads, and tombstone deletions are distributed across different nodes in the cluster rather than bottlenecking a single coordinator.

Required Changes to the GET /listings API:

To support year-based partitioning cleanly, the GET /listings endpoint should be updated to accept an optional temporal filter:

- **• Endpoint Signature:** Support GET /listings?year=2026 or date-range parameters GET /listings?from_year=2025&to_year=2026.
- **• Default Browsing Behavior:** When a client requests GET /listings without parameters, the handler defaults to the current calendar year (time.Now().UTC().Year()). This ensures fast, low-latency lookups for active inventory without performing broad scans.

How the Handler Would Find Listings Across Years:

Because Cassandra hashes partition keys using the Murmur3Partitioner, partition keys are scattered randomly around the ring; Cassandra does not support cross-partition range queries (e.g. 'WHERE year >= 2025') in a single SELECT query. Therefore, to query listings across a multi-year span [start_year, end_year], the handler must follow these steps:

- **1. Partition Key Derivation:** The handler determines the set of integer years spanning the interval (e.g. [2024, 2025, 2026]).
- **2. Concurrent Per-Partition Queries:** The handler issues separate targeted queries: 'SELECT ... FROM available_listings WHERE year = ?' for each year in parallel using Go goroutines and sync.WaitGroup.
- **3. In-Memory Merging and Sorting:** Because each partition is already ordered internally by listed_at ASC, the handler aggregates the listing slices from each year query, merges them in memory, sorts the combined slice by listed_at, and serializes the result into JSON.

Section IV: Tunable Consistency and Distributed Transactions

Question IV.1: In-Depth Analysis of the Five Consistency Configurations

Background and Theoretical Consistency Model:

In this project, the keyspace marketplace_history is defined with replication factor N = 3 in datacenter1. The Cassandra consistency levels operate as follows:

- **• Consistency ONE:** Requires confirmation from exactly 1 replica node.
- **• Consistency QUORUM:** Requires confirmation from a strict majority: floor(N / 2) + 1 = floor(3 / 2) + 1 = 2 replica nodes.
- **• Consistency ALL:** Requires confirmation from all N = 3 replica nodes.

- • **Strong Consistency Theorem (W + R > N):** Strong consistency (read-your-writes consistency) is mathematically guaranteed when the write quorum W and read quorum R satisfy W + R > N (here, W + R > 3). Under this condition, the write replica set and read replica set must overlap in at least one node, ensuring the read sees the latest committed write.

Below is the systematic analysis for each of the five experimental runs conducted with run_experiment.sh:

Case 1: QUORUM / QUORUM with 3 Available Nodes (Reference Baseline)

- • **Cluster State:** All 3 Cassandra nodes (cassandra-1, cassandra-2, cassandra-3) are operational and report 'UN' in nodetool status.
- • **Consistency Level Analysis:** Write Consistency = QUORUM (requires 2 nodes). Read Consistency = QUORUM (requires 2 nodes). Condition: W + R = 2 + 2 = 4 > 3 = N. Strong consistency is guaranteed.
- • **Observed Output (out/experiment-write-quorum-read-quorum-nodes-3.txt):** HTTP/1.1 200 OK. Alice's account shows balance = 50, listings array contains 5 sold items, sales array contains 5 completed sales, and orders array is empty [].
- • **System Logs and Database Inspection:** Marketplace service logs show clean sequential projection for all 5 listings and 5 orders: 'listing listing-X committed in Postgres and projected to Cassandra' and 'order order-X committed in Postgres and projected to Cassandra'. Queries against Postgres confirm 5 sold listings, 5 completed orders, Bob balance = 50, and Alice balance = 50. Cassandra tables contain matching records in listings_by_seller and sales_by_seller.
- • **Agreement with Consistency Guarantees:** The observed result strictly agrees with the strong consistency guarantee. Postgres and Cassandra views are fully synchronized.

Case 2: QUORUM / QUORUM with 2 Available Nodes (Fault-Tolerant Quorum)

- • **Cluster State:** Only 2 nodes are active (cassandra-1, cassandra-2); cassandra-3 is not started. Nodetool status reports 2 nodes as 'UN'.
- • **Consistency Level Analysis:** Write Consistency = QUORUM (requires 2 nodes). Read Consistency = QUORUM (requires 2 nodes). Because 2 nodes are online, the quorum threshold of 2 is fully satisfied (2 >= 2). Overlap is guaranteed on the 2 surviving replicas (W + R = 4 > 3).
- • **Observed Output (out/experiment-write-quorum-read-quorum-nodes-2.txt):** HTTP/1.1 200 OK. Alice's account displays balance = 50, 5 sold listings, and 5 completed sales.
- • **System Logs and Database Inspection:** Marketplace logs confirm zero projection failures. Both Postgres and Cassandra acknowledge all mutations without error.
- • **Agreement with Consistency Guarantees:** Strictly agrees. This experiment demonstrates Cassandra's fault tolerance under majority quorum: a cluster with replication factor N = 3 can tolerate the complete failure of F = 1 node without sacrificing either strong consistency or availability.

Case 3: ONE / ONE with 2 Available Nodes (Weak Consistency)

- • **Cluster State:** 2 nodes are active (cassandra-1, cassandra-2). Nodetool reports 2 'UN' nodes.
- • **Consistency Level Analysis:** Write Consistency = ONE (requires 1 node). Read Consistency = ONE (requires 1 node). Condition: W + R = 1 + 1 = 2 <= 3 = N. Strong consistency is NOT guaranteed; the system operates under eventual consistency.
- • **Observed Output (out/experiment-write-one-read-one-nodes-2.txt):** HTTP/1.1 200 OK. Alice's account shows balance = 50, 5 sold listings, and 5 sales.

- • **Analysis of Guarantees vs. Observations:** Although the output shows all 5 sales, this consistency is incidental rather than guaranteed by the database. In our local test environment where requests execute sequentially over Docker bridge networking with low latency, background gossip and inter-node mutation forwarding finish within milliseconds. However, mathematically W + R <= N allows stale reads: if a write confirms on node 1 while node 2 is momentarily delayed, a read at ONE hitting node 2 would return an empty or partial view without raising an error.

Case 4: ALL / ONE with 2 Available Nodes (Write Availability Failure & Data Divergence)

- • **Cluster State:** 2 nodes are active (cassandra-1, cassandra-2); cassandra-3 is absent.
- • **Consistency Level Analysis:** Write Consistency = ALL (requires all 3 replicas). Read Consistency = ONE (requires 1 replica).
- • **System Behavior & Log Evidence:** During listing and order creation, the PostgreSQL transaction commits successfully first. Then marketplace-service invokes history.WriteListing and history.WriteOrder to project to Cassandra with consistency ALL. Because only 2 nodes are online, Cassandra fails to satisfy ALL and throws: 'UnavailableException: Cannot achieve consistency level ALL (3 required, 2 alive)'. In handlers.go, the service catches the Cassandra error, logs 'ERROR listing/order committed in Postgres but Cassandra projection failed', but continues execution and returns HTTP 201 Created to the caller because Postgres already committed.
- • **Observed Output (out/experiment-write-all-read-one-nodes-2.txt):** HTTP/1.1 200 OK. Alice's account payload displays: balance = 50, listings = [], orders = [], sales = [].
- • **Inconsistency Explanation:** The observed data exhibits severe inconsistency: Alice has an authoritative balance of 50 credits (earned from sales), but her listings and sales arrays are completely empty. The read at consistency ONE succeeded because 2 nodes were alive, but because all writes to Cassandra failed, the Cassandra tables hold zero rows.
- • **Verifying True Order Completion:** We know the orders completed successfully despite the empty Cassandra views because: (1) Direct SQL inspection of PostgreSQL ('SELECT * FROM orders') reveals all 5 orders are completed; (2) Bob's balance was debited by 50 and Alice's balance was credited by 50 in Postgres; (3) Marketplace logs explicitly confirm 'order order-X committed in Postgres but Cassandra projection failed'; and (4) Each POST /orders request returned HTTP 201 Created with a valid order payload.

Case 5: ONE / ALL with 2 Available Nodes (Read Availability Failure)

- • **Cluster State:** 2 nodes are active (cassandra-1, cassandra-2); cassandra-3 is absent.
- • **Consistency Level Analysis:** Write Consistency = ONE (requires 1 node). Read Consistency = ALL (requires all 3 nodes).
- • **System Behavior & Log Evidence:** During listing and checkout creation, writes to Cassandra succeed under consistency ONE because 2 alive nodes satisfy ONE. The service logs: 'INFO order order-X committed in Postgres and projected to Cassandra'. However, when the final request GET /accounts/Alice is received, accountHandler reads balance 50 from Postgres, and then calls history.ListingsBySeller with consistency ALL. Because ALL requires 3 nodes and only 2 are active, Cassandra aborts the query with 'UnavailableException: Cannot achieve consistency level ALL'. In handlers.go, writeHistoryError catches the failure, logs 'ERROR Cassandra view read failed (HTTP 503)', and returns HTTP 503 Service Unavailable.
- • **Observed Output (out/experiment-write-one-read-all-nodes-2.txt):** HTTP/1.1 503 Service Unavailable with body {"error":"history service unavailable"}.

- • **Agreement with Consistency Guarantees:** Agrees with Cassandra's availability trade-off: requiring ALL for reads on a degraded cluster sacrifices read availability.
- • **Verifying True Order Completion:** Even though the read endpoint returns HTTP 503, the orders completed successfully: (1) PostgreSQL contains all 5 orders marked 'completed' and balances reflect the transfers; (2) Direct cqlsh queries against Cassandra at consistency ONE confirm that all 5 orders and sales were successfully written during the write phase; and (3) The 503 error is exclusively a read-path failure caused by an overly stringent read consistency requirement.

Summary Comparison Table Across Consistency Configurations:

Case #	Nodes	Write / Read Level	W + R > N?	HTTP Status	Observed Balance	Observed Views	System Behavior & Consistency State
Case 1	3	QUORUM / QUORUM	4 > 3 (Yes)	200 OK	50	5 listings, 5 sales	Strongly consistent. Reference run with zero errors.
Case 2	2	QUORUM / QUORUM	4 > 3 (Yes)	200 OK	50	5 listings, 5 sales	Strongly consistent. Demonstrates 1-node fault tolerance.
Case 3	2	ONE / ONE	2 <= 3 (No)	200 OK	50	5 listings, 5 sales	Eventual consistency. Incidental match due to low latency.
Case 4	2	ALL / ONE	4 > 3 (N/A)	200 OK	50	Empty ([])	Data divergence. Cassandra writes fail; Postgres commits.
Case 5	2	ONE / ALL	4 > 3 (N/A)	503 Error	N/A	Service Unavailable	Read availability loss. Writes succeed; reads fail ALL.



Figure 2: Execution Evidence for QUORUM Baseline, ALL/ONE Projection Failures, and System Logs.

Question IV.2: Feasibility of Combining Cassandra Writes in PostgreSQL Transactions

Proposal Under Review:

In createOrderHandler (marketplace-service/handlers.go), writes to Cassandra tables may fail, causing materialized read views to diverge from Postgres. Someone proposes including history.WriteOrder directly inside the PostgreSQL transaction block (store.CreateOrder in marketplace/orders.go), intending that if either database operation fails, the transaction rolls back and the order will not complete.

Will this approach work?

No, this approach will not work. It is architecturally flawed and introduces severe data corruption risks:

- **1. Incompatible Transaction Domains (Absence of Two-Phase Commit):** PostgreSQL transactions (tx.Begin, tx.Commit, tx.Rollback) operate strictly within PostgreSQL's internal Write-Ahead Log (WAL) and MVCC storage engine. PostgreSQL has no awareness or control over external systems like Apache Cassandra. Placing a Go function call to Cassandra inside a Postgres transaction block does not make Cassandra part of the atomic transaction.
- **2. The Phantom Write Hazard (Irreversible Cross-Database Inconsistency):** Suppose history.WriteOrder is called before tx.Commit(). If Cassandra successfully writes the purchase and sale rows, but tx.Commit() subsequently fails in PostgreSQL (due to a serialization anomaly, unique constraint violation, connection timeout, or server crash), PostgreSQL rolls back. The order is cancelled, and account balances are unchanged in Postgres. However, Cassandra has already committed the write! Because Cassandra does not support distributed multi-database rollbacks, Cassandra will permanently retain records of an order and sale that never legally occurred in Postgres, creating phantom data corruption.
- **3. Severe Lock Contention and Connection Pool Starvation:** In store.CreateOrder, PostgreSQL holds exclusive row-level locks on the accounts and listings tables ('FOR UPDATE'). Cassandra writes involve distributed network RPCs across multiple nodes with potential retry timeouts of several seconds. Holding open critical PostgreSQL database locks while awaiting remote network I/O from Cassandra will cause thread blockages, cascading query delays, and database connection pool exhaustion across concurrent users.
- **4. Proper Industry Solution (Transactional Outbox Pattern):** The robust, cloud-native pattern for synchronizing relational operational databases with NoSQL read models is the Transactional Outbox

Pattern with Change Data Capture (CDC). An outbox event is written atomically to a dedicated Postgres outbox table within the same local transaction as the order. An asynchronous background consumer (such as Debezium with Kafka) reads the committed outbox stream and projects updates to Cassandra with idempotent retries, guaranteeing reliable eventual consistency without distributed transaction hazards.